

Entrega 2 - Algoritmos e Lógica

Grupo 1: Olívia Domingues, Giovanna Bueno, Gabriel Carrocci

Turma: CCOMP 1º Semestre Noturno

Laboratório Mal Assombrado - Estrutura do código

Para organização de responsabilidades dentro do código, fácil entendimento e evitar repetir código, o programa foi dividido em várias funções, as quais cada têm um objetivo específico para o funcionamento do algoritmo:

Lore() {

A função Lore() com o intuito de apresentar o contexto do jogo ao player. Ela é basicamente uma série de prints contando essa história e personalizando a experiência do usuário.

```
// função para apresentar o contexto ao jogador
// reference
public static void Lore()
{
    Console.WriteLine("[Você] Zzzzzzzzz...");
    Console.ReadLine();
    Console.WriteLine("[Rádio] EII, ACORDA!");
    Console.ReadLine();
    Console.WriteLine("[Você] 00");
    Console.ReadLine();
    Console.WriteLine("[Rádio] Parece que você dormiu no laboratório...Qual é o seu nome?");
    string nomeJogador = DefinirNomeJogador();
    Console.WriteLine($"[Rádio] Certo, {nomeJogador}...Como você pode perceber já anoiteceu. Para o seu azar, esse laboratório é assombrado pelo fantasma de um professor que tinha muito apego por esse lugar há muuuuitos anos. Só ele pode deixar você sair daí..");
    Console.ReadLine();
    Console.WriteLine($"[{nomeJogador}] E como que eu saio daqui?!");
    Console.ReadLine();
    Console.WriteLine($"[Rádio] Você tem que fazer uma prova, mas cuidado... A cada questão que você erra, ele suga pontos da sua energia vital até você ficar sem.. Se você acertar tudo, ele te deixa sair. Você pode tentar quantas vezes quiser, mas cuidado para não deixar ele sugar todos os pontos da sua energia.");
    Console.ReadLine();
    Console.WriteLine($"[{nomeJogador}] E se eu não souber nada?!");
    Console.ReadLine();
    Console.WriteLine($"[Rádio] Você pode desistir a qualquer momento da prova ou nem fazer-la, mas você terá que esperar amanhecer o dia para alguém destrancar o laboratório. Bom, eu não recomendo desistir, você não gostaria de passar a noite com um fantasma te rodeando, mas enfim você quem sabe...");
}
```

* Chama a função DefinirNomeJogador() para definir e validar o nome do player.

}

DefinirNomeJogador(): string {

Essa função, como o nome diz, tem o objetivo de definir e validar o nome do jogador de acordo com as seguintes regras:

```
// função para validar e definir nome do jogador
1 reference
public static string DefinirNomeJogador()
{
    Console.WriteLine("\nDigite seu nome: ");
    string nomeJogador = Console.ReadLine();

    while (string.IsNullOrEmpty(nomeJogador) || nomeJogador.Any(char.IsDigit) || nomeJogador.Any(char.IsPunctuation))
    {
        Console.WriteLine("O nome não pode ser vazio, conter números ou pontuações!\nDigite seu nome novamente: ");
        nomeJogador = Console.ReadLine();
    }

    return nomeJogador;
}
```

1. O programa pede ao usuário o seu nome e atribui à variável “nomeJogador”;
2. Ele entra no loop while() e ficará pedindo o nome do jogador novamente se:
 - a. O nome do jogador for vazio ou conter espaço;
 - b. Se o nome do jogador conter um dígito;
 - c. Se o nome do jogador conter pontuação ou caracteres especiais.
3. Se todas as condições anteriores forem falsas, ele retorna o nome do jogador.

```
}
```

DefinirResposta(): string{

Essa função tem o objetivo de definir e validar a resposta das perguntas.

```
// função para validar e definir a resposta
3 references
public static string DefinirResposta()
{
    Console.WriteLine("Escolha entre a, b ou c.\nDigite sua resposta: ");
    string resposta = Console.ReadLine().ToLower();

    string[] alternativasValidas = { "a", "b", "c" };
    while (!alternativasValidas.Contains(resposta))
    {
        Console.WriteLine("\nResposta inválida! Escolha entre a, b ou c: ");
        resposta = Console.ReadLine().ToLower();
    }

    return resposta;
}
```

1. Cria-se um array com as alternativas válidas: a, b ou c;
2. Enquanto a resposta do usuário não estiver dentre as alternativas válidas, o programa ficará pedindo a resposta novamente;
3. Se a condição anterior for/se tornar falsa, a resposta é retornada.

Desistir(): bool{

Essa função verifica se o usuário quer desistir ou não.

```
// função para verificar se o jogador deseja desistir
3 references
public static bool Desistir()
{
    Console.WriteLine("\nDeseja desistir?\n[1] Sim\nPressione qualquer outra tecla para continuar");
    string input = Console.ReadLine();

    if (input == "1")
    {
        Console.WriteLine("[Rádio] Você desistiu..Bom, boa sorte para passar a noite!");
        return true;
    }
    return false;
}
```

1. O programa pergunta se o usuário quer desistir. (1 para sim, qualquer outra tecla para não);
2. Se o usuário quiser, o programa imprime uma mensagem e retorna true, senão retorna false;

}

AtualizarPontuacaoErros(): (int, int) {

Essa função serve para atualizar as variáveis pontos e erros. Ele pede como parâmetros essas variáveis, a resposta certa da questão e a resposta do jogador:

```
// função para contabilizar pontos e erros do jogador
3 references
public static (int, int) AtualizarPontuacaoErros(string respostaCerta, string respostaJogador, int pontos, int erros)
{
    if (respostaJogador != respostaCerta)
    {
        erros++;
    }
    else
    {
        pontos++;
    }

    return (pontos, erros);
}
```

1. Se a resposta do jogador estiver errada, ou seja, for diferente da resposta certa, a variável erros é contabilizada com +1.
2. Senão, ele ganha um ponto;
3. Ao fim, a função retorna as variáveis atualizadas.

}

Fluxo principal (Main) {

A Main tem a responsabilidade de juntar toda a lógica do programa e definir o fluxo que o algoritmo seguirá de fato.

1. Antes de tudo, o programa chama a função Lore para contextualizar o player sobre o jogo.
2. Inicializa-se a variável energia com o valor 10;

```

while (true)
{
    int pontos = 0;
    int erros = 0;

    if (Desistir())
    {
        break;
    }
}

```

3. Em seguida, o programa entra em um loop while true para ser quebrado de acordo com determinadas condições de parada;
4. No começo de toda nova repetição, as variáveis pontos e erros são zeradas;
5. No começo de toda repetição, o programa confere se o player deseja desistir. Se sim, o loop é quebrado;

```

// Pergunta 1
Console.WriteLine("***

[QUESTÃO 01]
Quem foi a primeira pessoa a viajar ao Espaço?
    [a] Yuri Gagarin
    [b] A cadela Laika
    [c] Neil Armstrong
***");
string resposta = DefinirResposta();
(pontos, erros) = AtualizarPontuacaoErros("a", resposta, pontos, erros);
if (Desistir())
{
    break;
}

// Pergunta 2
Console.WriteLine("***

[QUESTÃO 02]
Quais são os respectivos valores de X e Y do Sistema de Equações abaixo?
    {X + Y = 28
    {2X + Y = 18

    [a] X = 4, Y = 5
    [b] X = 3, Y = 2/5
    [c] X = 10, Y = 10
***");
resposta = DefinirResposta();
(pontos, erros) = AtualizarPontuacaoErros("c", resposta, pontos, erros);
if (Desistir())
{
    break;
}

// Pergunta 3
Console.WriteLine("***

[QUESTÃO 03]
A Luz do Sol demora quando tempo para chegar à Terra?
    [a] 1 minuto
    [b] 8 minutos
    [c] 5 minutos
***");
resposta = DefinirResposta();
(pontos, erros) = AtualizarPontuacaoErros("b", resposta, pontos, erros);

```

6. Em seguida, o programa tem 3 perguntas, todas seguem a mesma estrutura:
 - a. Chama a função DefinirResposta();

- b. Passa e reatribui as variáveis pontos e erros, passa a resposta certa da questão e a resposta do jogador a função AtualizarPontuacaoErros();
- c. Verifica se o player deseja desistir a cada função;

```
// se o jogador acertar tudo, ele ganha e o programa acaba
if (pontos == 3)
{
    Console.WriteLine("[Fantasma] Parabéns, você conseguiu...Pode sair");
    break;
}

energia -= erros;

// se a energia do jogador acabar, ele perde e o programa acaba
if (energia <= 0)
{
    Console.WriteLine("Sua energia acabou...");
    break;
}

Console.WriteLine($"""
Você perdeu {erros} de energia
| Energia: {energia}
""");
}
```

- 7. Ao fim das questões, o programa verifica se o player acertou as 3 questões. Se sim, ele ganha e o programa acaba, senão:
- 8. O programa atualiza a energia do jogador, subtraindo a quantidade de erros.
- 9. Depois ele verifica se a energia for menor ou igual a 0. Se sim, o player perde e o programa acaba, senão:
- 10. O programa printa o quanto o jogador errou e o quanto de energia ele ainda tem.

}